

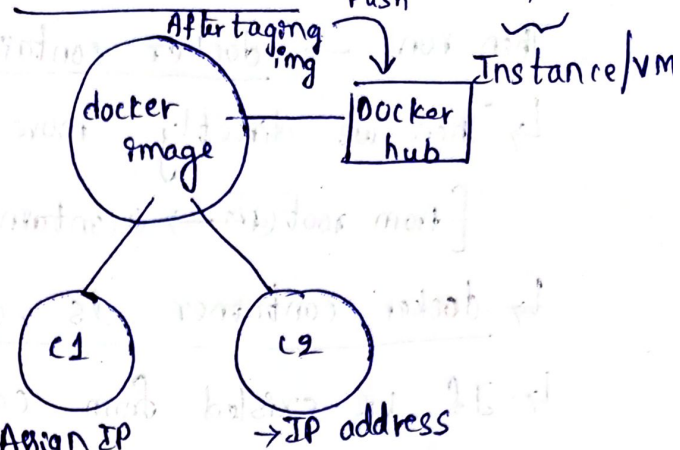
11/10/24

DOCKER

Virtualization



Containerisation



- Assign IP address
- Assigns container Id
- Containers run on required space
- While running container we get the terminal to run all programs.

→ Docker host - Install docker

→ Once container is running we can attach container to network, volumes. Container updating with image

→ Docker hub - public repository - store images (all customised) [We can download imgs from docker hub (nginx)]

Basic commands of Docker [CMD-level]

1) docker --version → To check the docker version

2) apt install docker.io → To install docker

3) docker image ls -a → Displays docker image

4) To download docker image from docker hub.

Command is docker pull nginx (any image)

↳ docker image ls -a
nginx | Tag latest | image id | displays created size

5) Run docker container → check for containers

Command → docker container ls -a

Then run → docker container run -it nginx /bin/bash

↳ Then we directly move to container

[from root@vm → @containerid #]

↳ docker container ls -a → to check the container

↳ If we existed from container using exit ←

Then it shows container in exited state.

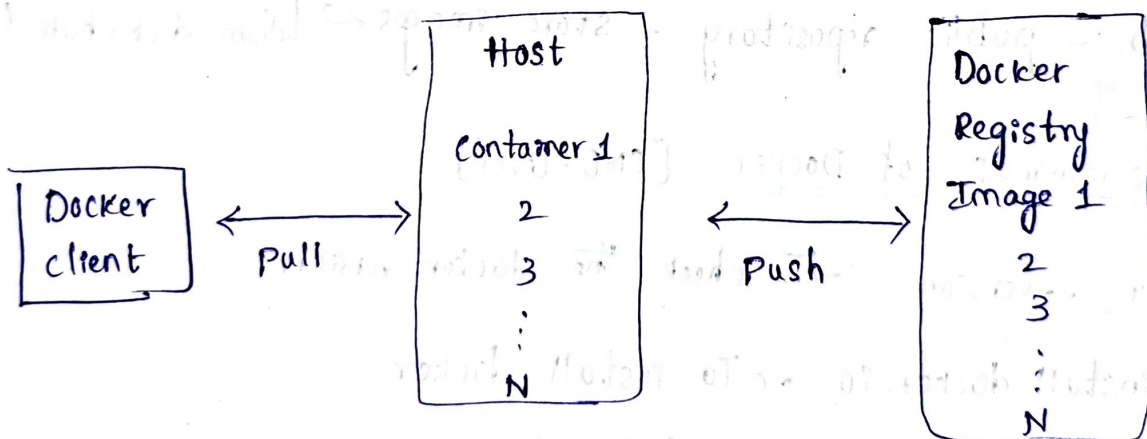
6) docker container attach containerId

docker container start containerId ↘

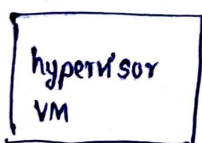
docker container attach containerId ↘

ls -li

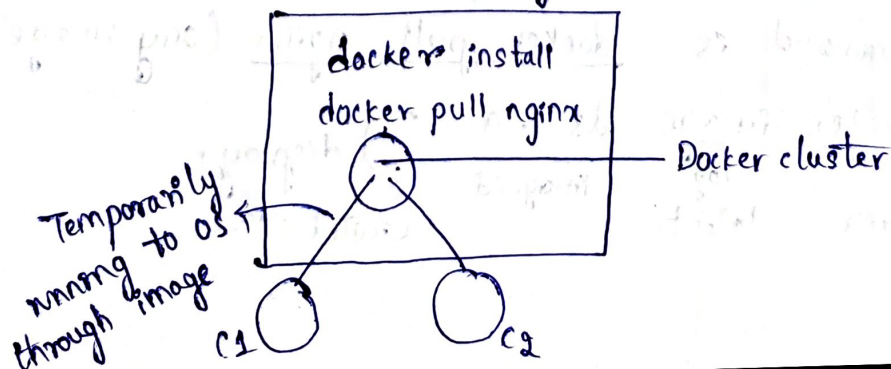
→ Container is running instance of an image. Manage container through API & GUI



virtualisation



UBUNTU OS / any OS.



→ Again inside container we have OS.

→ Once we install docker we have 2 Network interfaces.

1) Physical adapter

2) Docker network adapter

→ Range for IP address 172.16.10.1 , 172.16.10.2

→ Docker 0 network interface gives the IP address, for the containers.

→ Docker container tool is best for load balancer.

Load balancer — Distributes load while traffic occurs.

* If traffic is more Docker 0 automatically scale up the containers. [like ~~C1~~ → C1, C2, C3].

* To save all changes in docker image then we need to customize our own docker image & same image is shared into docker hub to save, so that other users can pull the image from docker hub. (Public repository)

7) ifconfig → shows all IP addresses

8) docker container rm containername/id [To remove container]

9) docker container stop containername/id [Stops container and we came out of container]

10) docker container start containername/id [container will start]

11) To pause a container → docker container pause containerId

↳ To start docker container start containerId

12) apt-get update

13) apt install git

14) apt install tree

15) Exporting docker container into the file → file
docker container export containerid > devops.tar

16) Importing image to convert tarfile into image
docker image import devops.tar imagename

↳ docker image ls -a

17) To upload image to docker hub, we need to set a tag
docker image tag ^{local} dockerimageid dockerhubid/newname (or same image name)

If we don't give version it takes as latest by default. version

→ docker push ^(username) dockerhubid/imagename:1.0

If we get denied access then use docker login and do push.

[In local machine storage increases & when it is pushed into dockerhub then storage compresses]

→ To pull image from another vm the command is

docker pull dockerhubid/imagename:version

check docker image downloaded or not docker image ls -a

→ docker container run -it dockerhubid/imagename:version /bin/bash

Docker Network

docker 0 - docker network

ens 33 - Physical network

- 1) docker container run -d -p 3600:80 --name containername
imagename
-p → port [80 port is enabled]
- 2) docker network ls displays how many n/w are running
Default - Bridge [n/w communication for docker]
- 3) docker network inspect bridge [Detail explanation of network]

Docker file

Docker can build images automatically by reading the instructions from Docker file. It is a text document that contains all commands a user could execute all command line to assemble an image.

(editor tool)

- 1) Create a docker file → nano Dockerfile

In that editor commands → FROM Ubuntu:16.04

(Image will automatically download in vm host)

- 2) docker image build -t imagename:version

In same file

RUN apt-get update && apt install apache2 -y

RUN touch /tmp/test [creating a file here]

RUN touch /tmp/test 1

{none}

[update, apache-2 get installed while building image]

Again build the image with same image or with new image also.

Docker compose is used for creating web applications.

It is a tool for running multiple container application on docker defined using compose file format.

A compose file is used to define how one or more containers that make up your application are configured once you have a compose file, you can create and start your application with a single command: docker compose up.

→ [docker container rm -rf]
on (stop & remove)

Commands Before creating delete all containers.

→ create one file nano docker-compose.yml

open this file

version: '3'

Services:

webapp1:

image: nginx

ports:

- "8000:80"

Save & run - docker-compose up -d

→ If docker compose not installed

apt install docker-compose

If any mismatchings apt-get update --fix-missing

→ Then webapp get created.

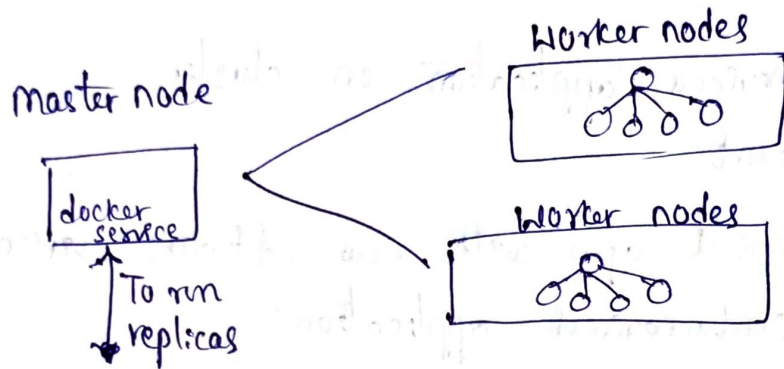
→ To see ip address of container - docker container inspect containerid

Then take ipaddress & paste in browser with :8000.

- docker-compose ps // displays how many processes/apps are running
- docker-compose start // To start containers

Docker Swarm

When network configure is heavy then in network: Master node it distributes the load to worker nodes.



Replicas - To run multiple container in background.

How to create docker swarm.

- docker swarm init (if already exists)
- docker swarm leave (slave)
- generate token then share that token on another VM.
- docker node ls (master vm) // displays docker swarm VMs [master (worker node)]
- docker service create replicas no.of replicas imagename(old/new) (master vm)
- docker service ls (worker Node vm)
- docker service scale pn=some number [ex-20] To increase replicas.
(Worker node) ← runs on
- docker container ls -a [master node] It will not work here